

Enabling Easy, Correct, and Reproducible Evaluation of Distributed Storage Systems

Shawn Zimmermann, Shengzhou Qiang, Zhuoyue Zhao, Haonan Lu



Background

Systems evaluation is to compare the performance, such as latency and throughput, of systems under test (SUTs) by conducting a set of experiments. Evaluation is complex, as it involves workload design or incorporation, the integration of workloads and SUTs, the design and control of experimental settings, such as the scale and placement of clients and servers, and the presentation of evaluation results.

Motivation

Mistakes in any part of the evaluation process may result in sub-optimal evaluations.

Sub-optimal evaluations can be caused by many easy to make mistakes such as:

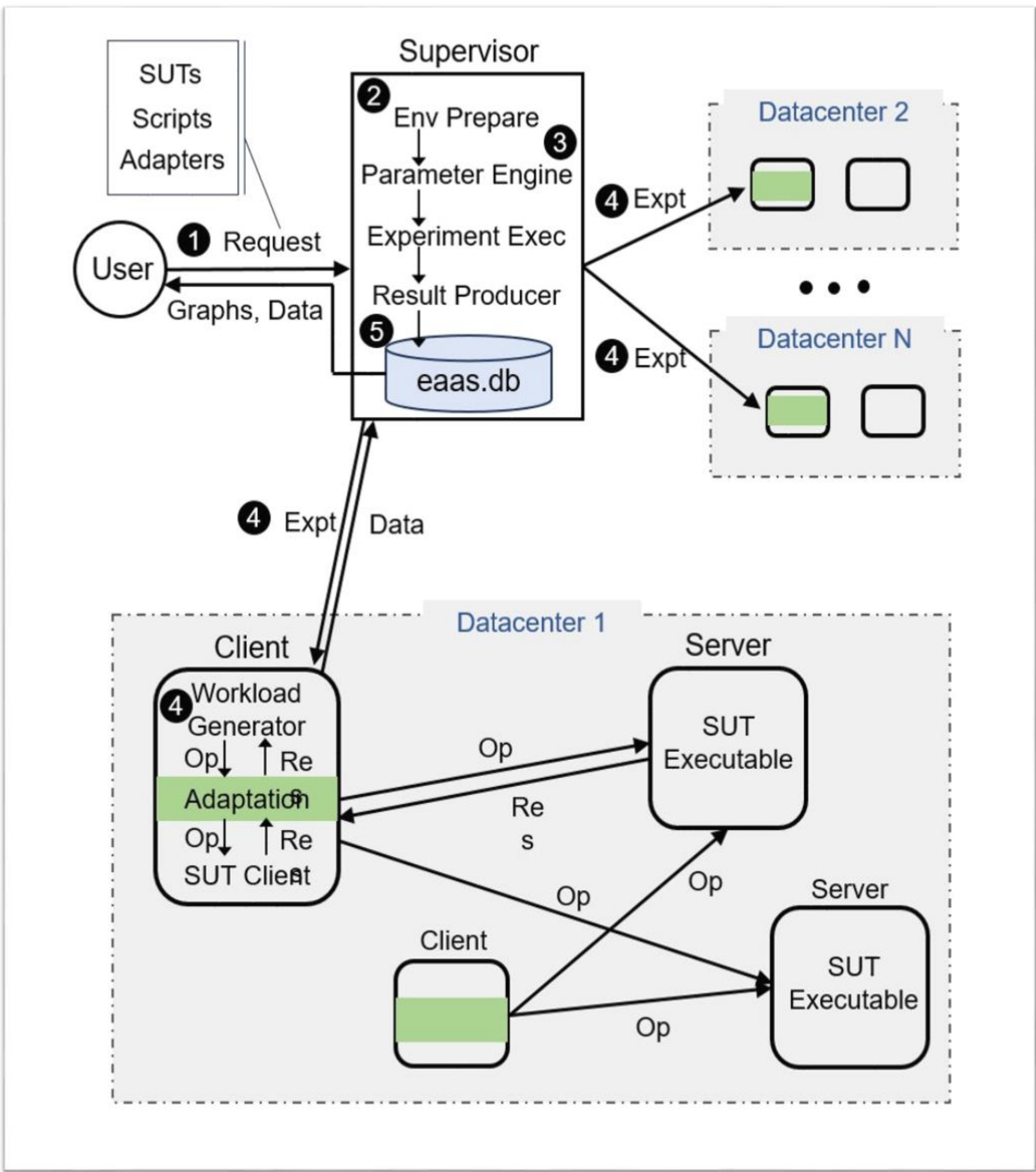
- Selective Workloads
- Restricted experimental settings
- Improper result presentation

Existing practices require manual effort to perform these complex tasks and need careful effort to avoid mistakes. **As a result existing evaluation methods are tedious and time consuming.**

These ad-hoc practices make experiments difficult to track and reproduce, as variations in any detail of any task may result in different experimental results.

Evaluation Life Cycle [See Figure A]

1. The user starts an evaluation, providing all necessary scripts and configurations
2. The supervisor validates the request, sanity checking configurations, prepares the experimental environments and orchestrates the evaluation
3. The parameter engine generates a set of experiments by enumerating the control parameters
4. The clients generate workloads specified by the control parameters and execute the experiments
5. Once an experiment is complete, the supervisor stores its raw data and performance numbers in an evaluation database. Upon completion of each request, a set of initial evaluation graphs are generated.



A An overview of EaaS architecture and its evaluation life cycle

Design

Supervisor

Handles evaluation requests, prepares experimental environments, creates a broad set of experiments, coordinates experiments, and generates evaluation graphs.

Parameter Engine

Collects a set of control parameters that uniquely characterize each experiment, enumerates their values to generate a comprehensive set of experiments, and documents these parameters for easy reproduction of experiments.

Workload Generator

Ran on the client, generates the workload as specified by the experiments control parameter.

Adaptation Layer

Shim adaptation layer, includes three language adapters that enable cross-language compatibility for SUTs.

Result Producer

A set of Python scripts, to generate evaluation graphs.

Overhead Results

We define overhead as **the percentage of time spent in EaaS' adaptation layer over the overall latency**. Most adapter APIs have minimal overheads, less than 0.5%.

API Overheads			
Storage API	# Rows	Avg. Latency (ms)	Overhead (%)
GET	1	1.16	0.18
PUT	1	3.34	0.35
SET	1	1.68	0.15
DELETE	1	0.64	0.32
SCAN	1-50	2.41	0.31
REV_SCAN	1-50	2.54	0.24
BATCH_GET	10	3.26	20.58
BATCH_PUT	10	3.11	22.66

Future Work

Planned future directions include:

- Deployment on public clouds for completely hands-off evaluation services
- Unit testing and correctness testing capabilities